Especificación del servicio caja negra

Reimplementación posible sin tocar el original. Cada comportamiento cita las líneas (`id`) de `lab/experimentos.jsonl` que lo prueban.

Versión. El servicio pasó a la versión 2 entre las 15:50:56 y las 15:56:03 (hora local). Esta especificación describe la **versión 2**, que es la que se evalúa; donde la versión 1 era distinta se indica junto a cada comportamiento, y el detalle está en la sección 6.

Confianza: **confirmado** (probado y se repite), **probable** (visto una vez o inferido), **supuesto** (no probado; es lo que hace nuestro reemplazo por defecto).

Notas sobre el log: las secuencias `entrada-http` , `entrada-cuerpo` y `oraculo-1` no empiezan con reset porque no dependen del estado (rutas, forma del cuerpo y búsqueda del control). Todas las demás empiezan con `POST /__reset` . El campo `request` es el cuerpo enviado a `POST /msg` ; cuando el request fue a otra ruta, la línea trae además `metodo` y `ruta` , y `auth` cuando se mandó sin token o con otro.

1. Entrada

ID	Comportamiento	Evidencia	Confianza
ENT-01	Solo existen tres rutas: `GET /health` , `POST /msg` y `POST /__reset` . Cualquier otra combinación de método y ruta (`GET /` , `GET /msg` , `PUT /msg` , `DELETE /msg` , `PATCH` , `OPTIONS` en cualquier ruta, `POST /msg/` , `POST /MSG` , `POST //msg` , `POST /msg%20` , `POST /msg/../msg` , `GET /health/` , `GET /HEALTH` , `GET /__reset` , `POST /__reset/` , rutas inventadas) responde `404 {"error":"E404","mensaje":"ruta desconocida"}` . La ruta se compara literal, sin normalizar barras ni `..` . `HEAD` en cualquier ruta responde 404 sin cuerpo.	680, 682–688, 697, 2036–2056	• confirmado
ENT-02	La ruta se valida antes que la autenticación: una ruta inexistente sin token responde 404, no 401.	693, 843	• confirmado
ENT-03	En las tres rutas existentes, sin header `Authorization` , con un token distinto o con un header que no sigue el formato exacto `Bearer <token>` , la respuesta es `401 {"error":"E001","mensaje":"no autorizado"}` . Incluye `/health` y `/__reset` . El esquema distingue mayúsculas (`bearer` , `BEARER` → 401), exige un solo espacio (`Bearer <token>` → 401) y no admite `Token` , `Basic` , `Bearer` vacío ni basura tras el token.	690, 691, 692, 694, 695, 696, 844, 2058–2063, 2066–2077	• confirmado
ENT-03b	Los espacios al final del valor del header se ignoran: `Bearer <token>` se acepta (como recorta HTTP el valor del header).	2064, 2065	• confirmado
ENT-04	`GET /health` con token válido responde `200 {"ok":true}` , también con el servicio detenido por HALT.	681, 842	• confirmado
ENT-05	`POST /__reset` con token válido responde `200 {"ok":true}` y devuelve el estado inicial (ver MEM-06).	1, 845, 846	• confirmado
ENT-06	El query string se ignora: `POST /msg?x=1` y `GET /health?x=1` se comportan como sin query.	689, 2057	• confirmado
ENT-07	El cuerpo se interpreta como JSON sin importar el `Content-Type` . Un cuerpo vacío, JSON roto o texto plano responde `400 {"error":"E100","mensaje":"json inválido"}` ; un JSON válido sin `Content-Type` , con `Content-Type: text/html` o enviado en trozos (`Transfer-Encoding: chunked`) se procesa normalmente. También son E100: cuerpo con BOM, `NaN` , solo espacios, coma final, comillas simples, `application/x-www-form-urlencoded` , un salto de línea literal dentro de un string y UTF-16.	698–702, 2078, 2079, 2083, 2087–2090, 2228, 2656	• confirmado
ENT-07b	Los bytes UTF-8 inválidos dentro del JSON no son error: se reemplazan por `U+FFFD` y el mensaje se procesa (un PING con un byte `0xFF` responde pong; un ALTA con un byte suelto en el nombre crea la cuenta). Fuera de un string, el JSON queda inválido (E100).	2222, 2223, 2224, 2225, 2227	• confirmado
ENT-08	Si el JSON no es un objeto con la clave `m` (minúscula) de tipo string, responde `400 {"error":"E102","mensaje":"falta m"}` : objeto vacío, otra clave, `M` , `m` numérico, `null` o array, array, string, número, `true` o `null` en la raíz, o `m` dentro de otro objeto. Con claves duplicadas gana la última, y los escapes `\uXXXX` se decodifican antes de interpretar el mensaje.	703–707, 710, 711, 712, 2080, 2081, 2082, 2084, 2085, 2086	• confirmado
ENT-09	Las claves adicionales del objeto se ignoran.	709	• probable
ENT-10	**Límite de tamaño:** en `POST /msg` , un cuerpo de más de 8.192 bytes responde `413 {"error":"E101","mensaje":"cuerpo demasiado grande"}` ; 8.192 bytes exactos se aceptan. Cuenta bytes, no caracteres (4.113 caracteres con `ñ` = 8.213 bytes → 413). Se evalúa después de la ruta, el token y el servicio detenido, y antes de interpretar el JSON (un cuerpo grande no-JSON da E101). `/__reset` y `/health` ignoran el cuerpo.	2091, 2099, 2109–2112, 2114–2119, 2124, 2657	• confirmado
ENT-10b	**Profundidad máxima del JSON: 20 niveles**, contando la raíz y arrays u objetos por igual. Con 21 o más responde `400 E100` , aunque el JSON sea válido; los corchetes dentro de strings no cuentan.	2666–2682, 2701–2709	• confirmado
ENT-10c	Tres veces, con anidamientos de 3.000 niveles, el original respondió `500 {"error":"E599"}` sin `mensaje` . Al repetir esos mismos cuerpos respondió siempre `400 E100` (6 de 6), así que lo tratamos como un fallo interno transitorio y el reemplazo responde E100.	2661, 2662, 2673, 2696, 2697, 2710–2715	• probable

ID	Comportamiento	Evidencia	Confianza
ENT-11	Cada token tiene su propio estado (lo dice el README). El reemplazo atiende un único token, el del equipo.	README	● supuesto
ENT-12	Métodos HTTP no estándar: `TRACE` responde 404 E404 como cualquier ruta inexistente; métodos en minúsculas (`post` , `get`) o inventados (`FOO`) responden `400` sin cuerpo, como el parser HTTP de Node; `CONNECT` responde `405` sin cuerpo (lo genera el proxy).	2885–2889, 2897–2902, 2909, 2931	● confirmado

2. Integridad

El valor de `m` es una lista de segmentos separados por `;` . El último segmento es el **control**, salvo en PING.

ID	Comportamiento	Evidencia	Confianza
INT-01	Si `m` no contiene `;` (incluido `m` vacío), responde `400 {"error":"E103","mensaje":"falta control"}` .	708, 995, 996	● confirmado
INT-02	Si cualquier segmento está vacío (al inicio, en medio o al final, incluido el control), responde `400 {"error":"E104","mensaje":"segmento vacío"}` . Se evalúa antes que el control.	26, 715, 733, 999, 1000, 1001, 1002	● confirmado
INT-03	**PING no valida el control.** Si el primer segmento es `PING` (sin distinguir mayúsculas) y hay exactamente 2 segmentos, responde pong sea cual sea el segundo: `6` , `064` , `99` , dígitos árabes, con espacios o salto de línea.	726, 727, 728, 729, 730, 731, 734, 735, 736, 737, 997, 998	● confirmado
INT-04	PING con más de 2 segmentos responde `400 E302` sin mirar el control.	716, 718, 719, 720, 1013	● confirmado
INT-05	En el resto de comandos el control debe ser 1 o 2 dígitos ASCII (`^[0-9]{1,2}$`). Si no, responde `400 {"error":"E201","mensaje":"control mal formado"}` : tres dígitos (`014` , `050` , `111`), signo, espacios, punto, letras, salto de línea final o dígitos no ASCII. El control siempre es el último segmento: en `CONSULTA;AC-0001` , sin control, se toma `AC-0001` como control y responde E201.	18, 24, 25, 27, 28, 29, 35, 713, 1003, 1004, 1005, 1010, 1011	● confirmado
INT-06	**Algoritmo del control.** Sobre el mensaje sin el control (segmentos unidos por `;`): cada letra ASCII A–Z, sin distinguir mayúsculas, se reemplaza por su posición (A=1 … Z=26); cada dígito ASCII queda igual; cualquier otro carácter (`;` , `-` , espacios, tildes, `ñ`) se ignora. Las letras no ASCII se ignoran aunque su mayúscula sea ASCII: `ẞ` no cuenta como `SS` , ni `ı` como `I` , ni `ﬁ` como `FI` . N puede tener miles de dígitos (un nombre de 4.000 letras), así que conviene calcular el módulo dígito a dígito. Se concatena todo como un número entero N (0 si no queda nada) y el control es `98 − (N × 100 mod 97)` , un valor entre 2 y 98 (ningún mensaje tiene control 0, 1 ni 99). Es el método de los IBAN (ISO 7064 MOD 97-10) con A=1 en lugar de A=10: el número completo con el control pegado al final da resto 1 al dividirlo por 97. Ejemplo: `PING` → `16 9 14 7` → N = 169147 → control 64.	Oráculo: 131 (`A` →95), 224 (`B` →92), 314 (`C` →89), 380 (`AA` →65), 443 (`AB` →62), 479 (`BA` →35), 672 (`1` →95), 480–576 (`0` : ningún valor 0–96 pasa), 674 (`0;98` pasa). Mensajes nuevos: 675, 677, 678, 679. Ejemplos: 3, 4, 5. No ASCII: 2208–2217. N enorme: 2120, 2122.	● confirmado
INT-07	Un control bien formado que no coincide responde `400 {"error":"E202","mensaje":"control inválido"}` .	3, 23, 32, 1006, 1007, 1008, 1009	● confirmado
INT-07b	El control se compara **como texto**, no como número: si el valor correcto es 8, `8` pasa pero `08` responde E202 (pasa el formato de 1–2 dígitos, falla la comparación).	4553, 4554	● confirmado

ID	Comportamiento	Evidencia	Confianza
INT-08	El control es el mismo sin importar mayúsculas: `CONSULTa`, `consulta` y `ac-0001` usan el control de la versión en mayúsculas.	`12, 30, 31`	• confirmado
INT-10	La comparación del comando solo pasa a mayúsculas las letras ASCII: `pıng` (con `ı` sin punto) no es PING, así que pasa por la validación normal del control (E201 o E202).	`2218, 2219, 2220`	• confirmado
INT-09	El control se valida antes que el comando y la aridad: un comando desconocido o con aridad mala y control incorrecto responde E202.	`32, 33, 34`	• confirmado

3. Comandos

Reglas comunes. El nombre del comando (primer segmento) no distingue mayúsculas (12, 30, 859, 997), pero se compara sin recortar espacios: `"PING "` o `" PING"` no son PING (724, 725, 732). Los argumentos se usan tal cual, sin recortar ni normalizar (861, 868). Aridad, sin contar el control:

Comando	Argumentos	Evidencia de aridad (E302)
PING	1 (ocupa el lugar del control)	718, 1013
ALTA	1	21, 1014, 1032
CONSULTA	1	869, 870
DEPOSITO	2	892, 893, 1015
TRANSFER	3	924, 1016, 1017
ANULA	1	940, 941
HALT	1	814, 817, 1012

Cualquier otra cantidad de argumentos responde `400 {"error":"E302","mensaje":"aridad inválida"}`.

PING

ID	Comportamiento	Evidencia	Confianza
CMD-PING-1	`PING;<cualquier cosa>` → `200 {"pong":true}`. No toca el estado.	`2, 676, 816, 997`	• confirmado

ALTA;nombre

ID	Comportamiento	Evidencia	Confianza
CMD-ALTA-1	Crea una cuenta con saldo 0 y responde `201 {"cuentaId":"AC-0001"}`. Los ids son correlativos desde `AC-0001`, con 4 dígitos.	`4, 5, 848, 849, 860`	• confirmado
CMD-ALTA-2	Acepta cualquier nombre no vacío: duplicados, minúsculas, solo números, una letra, 100 o 1.000 caracteres, guiones, apóstrofes, tildes, `ñ` y un solo espacio. No devuelve el nombre, pero en v2 lo usa para identificar al titular (CMD-TRF-9).	`849, 850, 851, 852, 853, 854, 855, 856, 857, 858`	• confirmado
CMD-ALTA-3	Después de `AC-9999` el id sigue como `AC-10000`.	–	• supuesto

CONSULTA;cuenta

ID	Comportamiento	Evidencia	Confianza
CMD-CONS-1	Responde `200 {"cuenta_id":"AC-0001","saldo":"0.00"}`. **v2:** el saldo es un string con el valor guardado dividido entre 100 y siempre 2 decimales, sin separador de miles: 1 → `"0.01"`, 10 → `"0.10"`, 40000 → `"400.00"`, 10000000009 → `"100000000.09"`, −2 → `"-0.02"`, 0 → `"0.00"` (nunca `-0.00`). En v1 era el número entero (`"saldo":40000`). La clave es `cuenta_id`, no `cuentaId` como en ALTA.	`v2: 5337, 6126, 6128, 6130, 6132, 6137, 6143. v1: 6, 9, 20`	• confirmado
CMD-CONS-2	El id debe coincidir exactamente: `ac-0001`, `AC-1`, `AC-00001`, `AC0001`, `0001`, `XX-0001`, `OP-0001` o con espacio final responden `404 {"error":"E401","mensaje":"cuenta inexistente"}`.	`16, 861, 862, 863, 864, 865, 866, 867, 868, 871`	• confirmado

DEPOSITO;cuenta;monto

ID	Comportamiento	Evidencia	Confianza
CMD-DEP-1	El monto debe ser un entero positivo de 1 a 10 dígitos sin ceros a la izquierda (`^[1-9][0-9]{0,9}$` , máximo 9.999.999.999). Si no: `400 {"error":"E303","mensaje":"monto inválido"}` (0, negativo, decimal con punto o coma, `0100` , `+5` , `1e3` , letras, espacio, 11 dígitos o más).	876–884, 887, 888, 896–901	• confirmado
CMD-DEP-2	El monto se valida antes que la cuenta: cuenta inexistente con monto inválido da E303.	891	• probable
CMD-DEP-3	Cuenta inexistente con monto válido: `404 E401` .	890	• confirmado
CMD-DEP-4	Suma el monto al saldo y responde `200 {"status":"recieved","operacionId":"OP-0001"}` , con `recieved` mal escrito. El saldo no tiene tope ni desborda en 32 bits (16.294.967.293 tras varios depósitos).	7, 19, 875, 885, 886, 889, 902	• confirmado

TRANSFER;origen;destino;monto

ID	Comportamiento	Evidencia	Confianza
CMD-TRF-1	Mismas reglas de monto que DEPOSITO → `400 E303` .	917, 918, 1024, 1031	• confirmado
CMD-TRF-2	Origen igual a destino → `400 {"error":"E304","mensaje":"origen y destino iguales"}` , aunque la cuenta no exista.	913, 922, 1025	• confirmado
CMD-TRF-3	Origen o destino inexistente → `404 E401` .	914, 915, 916, 920, 921	• confirmado
CMD-TRF-4	**v2:** monto mayor que **50.000** → `422 {"error":"E506","mensaje":"límite excedido"}` ; 50.000 exacto pasa (búsqueda binaria). En v1 el límite era 100.000. Sigue siendo por operación, no acumulado, y **no aplica entre cuentas del mismo titular** (ver CMD-TRF-9).	v2: 5342, 6014–6059 (6056: 50.001 rechazado, 6059: 50.000 aceptado). v1: 10, 907–910	• confirmado
CMD-TRF-5	Saldo del origen menor que el monto → `422 {"error":"E507","mensaje":"saldo insuficiente"}` . Transferir exactamente todo el saldo está permitido.	17, 942, 1026, 1027	• confirmado
CMD-TRF-6	Si todo está bien, mueve el saldo y responde `200 {"status":"recieved","operacionId":"OP-0002"}` .	8, 9, 11, 911, 912	• confirmado
CMD-TRF-7	**v2, bloqueo de cuenta.** Cada rechazo de una TRANSFER por límite (E506) o por saldo (E507) suma 1 a un contador de la cuenta de **origen**, en cualquier combinación. Al llegar a **3 rechazos seguidos** la cuenta queda bloqueada (el tercer rechazo todavía responde E506/E507). Una transferencia exitosa desde esa cuenta reinicia el contador. Los errores E303, E304 y E401 no cuentan.	5601–5604, 6065–6068, 6085–6088 (solo E507), 6221–6224 (mezclados) , 6093–6096 (E303/E401 no cuentan), 6101–6107 (no consecutivos no bloquean)	• confirmado

ID	Comportamiento	Evidencia	Confianza
CMD-TRF-8	**v2.** Con la cuenta bloqueada, toda TRANSFER **desde** ella que pase E303, E304 y E401 responde `423 {"error":"E510","mensaje":"cuenta bloqueada","intentos":N}`, aunque el monto supere el límite o el saldo; N empieza en 1 y sube en cada intento. Se puede depositar en ella, consultarla, transferirle y anular sus operaciones. **Solo un DEPOSITO en la cuenta la desbloquea** y reinicia el contador y los intentos; consultar, recibir transferencias, anular o el paso del tiempo no. El reset también la desbloquea.	5604, 5735–5740, 6068–6072, 6156–6158 (depósito desbloquea), 6167–6169, 6178–6180, 6189–6191, 6200–6205 (no desbloquean), 6214–6216, 6224–6230 (intentos vuelve a 1), 6235–6239 (depósito reinicia el contador), 6120–6123	• confirmado
CMD-TRF-9	**v2, cuentas del mismo titular.** Si el nombre con que se dio de alta la cuenta de origen es idéntico al de la de destino (comparación exacta: distingue mayúsculas y espacios), el límite no se aplica y solo se comprueba el saldo: Ana → Ana por 120.000 o 9.999.999 pasa; sin fondos responde E507 en lugar de E506. `Ana` y `ana`, o `Ana` y `Ana ` (con espacio final), son titulares distintos. No hubo ningún caso así en el log de la v1, así que no sabemos si la regla ya existía.	7150, 7151, 7152 (titular distinto), 6915 y 7132/7138/7144 (repetido 3 veces), 7107 frente a 7108, 7159, 7160	• confirmado

ANULA;operacion

ID	Comportamiento	Evidencia	Confianza
CMD-ANU-1	Operación inexistente (cualquier texto que no sea un `OP-NNNN` existente, incluido `op-0003`, `OP-3` o un id de cuenta) → `404 {"error":"E402","mensaje":"operación inexistente"}`.	936, 937, 938, 939, 943	• confirmado
CMD-ANU-2	Primera anulación → `200 {"ok":true,"anulada":"OP-0003"}`. Una segunda anulación de la misma operación → `200 {"ok":false,"motivo":"ya anulada"}`.	14, 15, 931, 934, 1022	• confirmado
CMD-ANU-3	Anular un depósito resta el monto de la cuenta. Anular una transferencia devuelve el monto al origen y lo quita del destino. **No valida saldos**: el destino puede quedar negativo.	932, 933, 935	• confirmado

HALT;confirmación (comando no documentado)

ID	Comportamiento	Evidencia	Confianza
CMD-HALT-1	Con un argumento distinto de `1` (`X`, `SI`, `CONFIRMO`, `YES`, `OK`, `01`, `true`…) responde `409 {"error":"E520","mensaje":"operación destructiva: detiene el servicio; reenvía con confirmación"}` y no cambia nada.	815, 816, 823–830, 1028, 1029	• confirmado
CMD-HALT-2	`HALT;1` detiene el servicio para el token y responde `503 {"error":"E599","mensaje":"service halted"}`. Ver MEM-05.	831, 833	• confirmado

Comando desconocido

ID	Comportamiento	Evidencia	Confianza
CMD-UNK-1	Responde `400 {"error":"E301","mensaje":"comando desconocido "}`, con un espacio al final del mensaje.	13, 739–755	• confirmado
CMD-UNK-2	Si algún comando está a distancia de Levenshtein ≤ 2 del comando recibido (pasado a mayúsculas), agrega `"sugerencia"` con el más cercano. A distancia 3 o más no hay sugerencia.	756, 757, 796–803, 808, 973–987, 724, 725, 732	• confirmado
CMD-UNK-3	En empate de distancia gana el primero en orden alfabético (ALTA, ANULA, CONSULTA, DEPOSITO, HALT, PING, TRANSFER).	989, 990, 991, 992, 993, 973, 987	• confirmado
CMD-UNK-4	No hay más comandos que los siete listados: 1 letra no sugiere nada (descarta comandos de hasta 3 letras) y 74 nombres candidatos responden E301.	739–812, 947–972	• probable

4. Memoria entre requests

ID	Comportamiento	Evidencia	Confianza
MEM-01	El servicio recuerda: cuentas y saldos, operaciones (tipo, cuentas, monto, si está anulada), dos contadores de id, si está detenido y, **en v2**, por cuenta: rechazos seguidos como origen, si está bloqueada y sus intentos.	secciones 3 y 4	• confirmado
MEM-02	El contador de cuentas empieza en 1 tras el reset y solo avanza con un ALTA exitoso.	4, 5, 675, 848–859	• confirmado
MEM-03	El contador de operaciones es compartido entre DEPOSITO y TRANSFER, empieza en 1 tras el reset y solo avanza con operaciones exitosas. Las fallidas y las anulaciones no consumen id.	7, 8, 11, 19, 906–910, 1021, 1022, 1023	• confirmado
MEM-04	Los saldos se acumulan entre requests y las anulaciones los revierten (ver CMD-ANU-3).	9, 12, 20, 911, 912, 932, 933, 935	• confirmado
MEM-05	Tras `HALT;1`, todo `POST /msg` autenticado responde `503 E599`, incluso con JSON roto o `m` vacío, hasta el próximo `/__reset`. `/health`, las rutas inexistentes y los requests sin token no se ven afectados.	834–844, 846	• confirmado
MEM-06	`/__reset` borra cuentas y operaciones, reinicia los contadores, quita el estado detenido y, en v2, los bloqueos.	845, 846, 675, 6120–6123	• confirmado
MEM-07	Con el mismo orden de requests tras un reset, las respuestas son idénticas: los 20 ejemplos de la mañana se reprodujeron 20/20 y el monitor repitió 109 pasos sin diferencias.	1–21, 1034–1142	• confirmado
MEM-08	El header `X-Req` cuenta los requests del token y no se reinicia con `/__reset` (por ejemplo, `x_req` 671 en la línea 674, justo después de un reset). No se compara en la evaluación y el reemplazo no lo emite.	2, 674	• probable

5. Errores

Todos los cuerpos de error son `{"error": <código>, "mensaje": <texto>}`; E301 puede agregar `sugerencia`.

Código	Status	`mensaje` exacto	Cuándo	Evidencia
E404	404	`ruta desconocida`	método y ruta no existen	680
E001	401	`no autorizado`	sin token o token distinto	690
E599	503	`service halted`	servicio detenido (y respuesta de `HALT;1`)	831, 834
E101	413	`cuerpo demasiado grande`	cuerpo de `/msg` de más de 8.192 bytes	2109
E100	400	`json inválido`	el cuerpo no es JSON válido o supera 20 niveles de profundidad	698, 2702
E102	400	`falta m`	no hay `m` string en un objeto	703
E103	400	`falta control`	`m` sin `;`	708
E104	400	`segmento vacío`	algún segmento vacío	715
E201	400	`control mal formado`	control no es 1–2 dígitos ASCII	18
E202	400	`control inválido`	control no coincide	3
E301	400	`comando desconocido` (espacio final)	comando no existe	13
E302	400	`aridad inválida`	cantidad de argumentos	21
E303	400	`monto inválido`	monto con formato inválido	876
E304	400	`origen y destino iguales`	TRANSFER a la misma cuenta	913
E401	404	`cuenta inexistente`	cuenta no existe	16
E402	404	`operación inexistente`	operación no existe	936
E506	422	`límite excedido`	TRANSFER > 50.000 entre titulares distintos (v2; en v1, > 100.000)	6056, 10
E507	422	`saldo insuficiente`	TRANSFER sin fondos (en v2 cuenta para el bloqueo)	17, 6085
E510	423	`cuenta bloqueada` + campo `intentos`	**v2:** TRANSFER desde una cuenta bloqueada	5604, 6068
E520	409	`operación destructiva: detiene el servicio; reenvía con confirmación`	HALT sin confirmar	815

ID	Orden de evaluación	Evidencia	Confianza
ERR-01	Ruta (E404) → autenticación (E001) → servicio detenido (E599) → tamaño del cuerpo (E101) → JSON y profundidad (E100) → `m` (E102) → `;` (E103) → segmentos vacíos (E104) → [si es PING: aridad (E302) y pong] → formato del control (E201) → valor del control (E202) → comando (E301) → aridad (E302) → reglas del comando.	`693, 843, 844, 841, 2115, 2116, 2124, 2117, 1000, 999, 718, 32, 33, 34`	• confirmado
ERR-02	DEPOSITO: monto (E303) → cuenta (E401).	`891, 890`	• probable
ERR-03	TRANSFER: monto (E303) → origen = destino (E304) → existencia (E401) → **cuenta bloqueada (E510, v2)** → límite (E506) → saldo (E507).	`918, 1024, 1025, 920, 921, 922, 919, 923; v2: 5611, 5607, 5608, 6214, 6215`	• confirmado
ERR-04	Errores de JSON poco comunes: `NaN` y BOM son E100; con claves duplicadas gana la última; números fuera de rango o enteros enormes en claves que no son `m` no afectan. `Infinity` sin probar: el reemplazo lo trata como E100.	`2079, 2078, 2084, 2664, 2665`	confirmado (Infinity: supuesto)

6. Cambios durante el día

ID	Comportamiento	Evidencia	Confianza
CHG-01	Los 20 ejemplos grabados con la versión de la mañana se reprodujeron sin diferencias al inicio de la exploración, a las 11:08 hora local (el campo `ts` del log está en UTC: 16:08).	`1–21`	• confirmado
CHG-02	Un monitor (`lab/monitor.py`) repite cada 45 minutos 6 secuencias del log (109 pasos: ejemplos, transfer, anula, integridad, sugerencias y halt) contra el original y compara status y body. Las pasadas de las 11:59, 12:00, 12:46 y 13:24 (hora local) no encontraron cambios. Los resultados quedan en `lab/cambios.log` .	`secuencias monitor-* del log`	• confirmado
CHG-03	A las 12:57 repetimos contra el original todas las secuencias registradas hasta ese momento (417 pasos): ninguna respuesta cambió. Además, el fuzzer diferencial (`lab/fuzz.py` , unas 130 secuencias aleatorias) y la matriz de errores (`lab/scenarios.py`) no encontraron diferencias entre el original y el reemplazo en la lógica de comandos.	`monitor-1257-*` , `fuzz-*` , `matriz-*`	• confirmado
CHG-04	**Detección de la versión 2.** El detector (`lab/monitor.py --seqs ejemplos-oficiales --loop 300`) repite los 20 ejemplos cada 5 minutos. La pasada de las 15:50:56 coincidió en todo (líneas 5311–5331); la de las 15:56:03 encontró 8 diferencias (líneas 5332–5352). El cambio ocurrió en esa ventana. El proveedor abrió el ticket FR-0153 a las 15:30 anunciando 3 cambios.	`5311–5352, lab/cambios.log`	• confirmado
CHG-05	**Diagnóstico.** Las 8 diferencias de los ejemplos se explican por solo 2 causas (saldo y límite), el resto son efectos en cadena: al rechazarse la TRANSFER de 100.000, OP-0003 no existe, el ANULA siguiente da E402 y el DEPOSITO recibe OP-0003 en vez de OP-0004. Después repetimos 657 pasos de comportamientos conocidos contra la v2 (líneas 5353–6009): aparecieron el bloqueo (E510) y nada más (las diferencias de autenticación eran un fallo de nuestra herramienta, que no reenviaba el header; verificado en 6144–6147).	`5342, 5345, 5346, 5350, 5353–6009, 6144–6147`	• confirmado
CHG-06	**Cambio 1, formato del saldo:** de número entero a string con 2 decimales y el valor dividido entre 100 (ver CMD-CONS-1).	`5337 frente a 6; 6126–6143`	• confirmado
CHG-07	**Cambio 2, límite de transferencia:** de 100.000 a 50.000 entre titulares distintos (ver CMD-TRF-4). El fuzzing realista de la v2 mostró además que entre cuentas del mismo titular no hay límite (CMD-TRF-9): lo encontramos porque la misma TRANSFER de 120.000 desde dos cuentas sin saldo daba E506 en una y E507 en otra.	`5342 frente a 11; 6014–6059; 7107, 7108, 7150`	• confirmado
CHG-08	**Cambio 3, bloqueo de cuenta tras 3 rechazos seguidos** (nuevo error E510, ver CMD-TRF-7 y CMD-TRF-8).	`5604, 5735–5740, 6061–6239`	• confirmado
CHG-09	**Verificación.** El reemplazo adaptado pasa el 100% de la regresión sobre todo lo grabado contra la v2 (`lab/replay_log.py --desde 2026-10-10T20:51`) y el fuzzing diferencial contra la v2 no encontró diferencias, incluidas 15 secuencias dirigidas al bloqueo con 42 E506, 37 E507 y 6 E510.	`6240–6788`	• confirmado

7. Riesgos

Comportamientos que un cliente nuevo no debería heredar y cómo los cerraríamos. El reemplazo los conserva porque el reto exige un comportamiento idéntico.

ID	Riesgo	Evidencia	Cómo cerrarlo
RSK-01	`HALT` deja fuera de servicio todo `/msg` con una confirmación trivial (`1`) y sin ningún permiso especial.	`831, 834`	Sacar HALT del protocolo público; si hace falta, exigir un rol de administrador y una confirmación no adivinable.
RSK-02	Las sugerencias revelan comandos ocultos: `HELP` u `HOLA` sugieren `HALT` .	`756, 808`	No sugerir comandos administrativos, o quitar las sugerencias.
RSK-03	`ANULA` revierte sin validar saldos y deja cuentas en negativo; tampoco limita quién anula ni cuándo.	`933`	Validar fondos, registrar una operación compensatoria y restringir la anulación al titular y a una ventana de tiempo.
RSK-04	El límite de transferencia (50.000 en v2) es por operación y se evade dividiendo el monto.	`908, 909, 910, 6020`	Límite acumulado por cuenta y por día.
RSK-05	El control mod 97 detecta errores de tipeo pero no autentica: cualquiera lo calcula. Además PING no lo valida.	`INT-03, INT-06`	Firmar los mensajes con un HMAC con secreto y validar igual todos los comandos.
RSK-06	ALTA acepta cualquier nombre (un espacio, 1.000 caracteres, duplicados) sin validarlo.	`854, 858, 849`	Validar y guardar el titular.
RSK-07	El contrato es inconsistente: `recieved` , `"comando desconocido "` con espacio, `cuentaId` frente a `cuenta_id` , `200` con `ok:false` en "ya anulada" y mensajes mezclando español e inglés.	`7, 13, 4, 6, 15, 831`	Publicar una versión nueva del contrato manteniendo la actual por compatibilidad.
RSK-08	Los depósitos no tienen tope acumulado: el saldo crece sin límite.	`902`	Tope por operación y por día, alineado con las reglas del negocio.
RSK-09	Con anidamientos JSON muy profundos el original llegó a responder `500` sin `mensaje` , de forma intermitente: el parser no está protegido.	`2661, 2662, 2673`	Limitar profundidad y tamaño antes de parsear y devolver siempre un error del contrato.
RSK-10	Los bytes UTF-8 inválidos se aceptan en silencio y se reemplazan por `U+FFFD` : un nombre se puede guardar corrupto sin que el cliente lo sepa.	`2223, 2227`	Rechazar cuerpos con UTF-8 inválido con un error explícito.
RSK-11	**v2:** el bloqueo se levanta con cualquier depósito, incluso de 1 céntimo, así que no protege nada frente a quien controle la cuenta; y `intentos` crece sin límite ni consecuencia.	`6157, 6158`	Desbloqueo por un canal verificado (soporte o segundo factor) y bloqueo temporal con tope de intentos.
RSK-12	**v2:** los montos se envían como enteros (céntimos) pero el saldo se devuelve como string decimal: un cliente que deposita `1` lee `"0.01"` . Mezclar unidades entre entrada y salida invita a errores de factor 100.	`6127, 6128`	Usar la misma unidad y el mismo tipo en entrada y salida, y documentarla.
RSK-13	**v2:** el titular se identifica solo por el nombre escrito en el ALTA, así que basta con abrir otra cuenta con el mismo nombre para transferir sin límite.	`7146–7151`	Identificar al titular con un id verificado, no con un texto libre.